CSE 4345: Big Data Analytics

Week 11, Part 2 — Seminal Research & Case Study

Department of Computer Science & Engineering
Premier University, Chittagong

Semester 2026

Part 2 Overview — Week 11 (Course Finale)

Today's Agenda

1. Paper 1 — Armbrust et al. (2020): Delta Lake
2. Paper 2 — Zaharia et al. (2021): Lakehouse
3. Paper 3 — Akidau et al. (2015): The Dataflow Model
4. Case Study — Enterprise Lakehouse Migration in Production
5. Live Exercise

Seminal Papers Covered

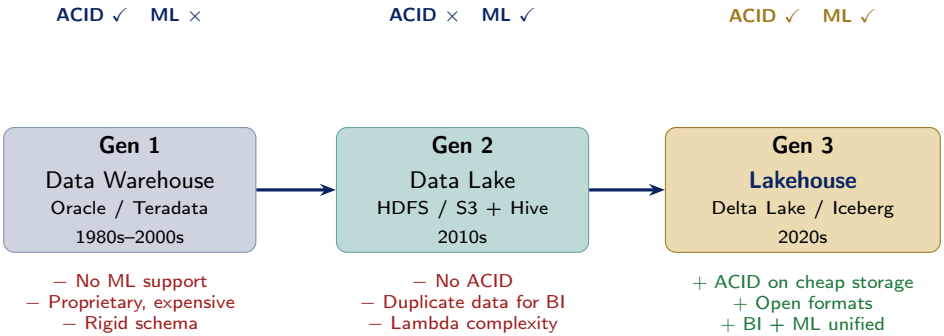
1. Armbrust et al. (2020). *Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.* VLDB 2020.
2. Armbrust, Ghodsi, Xin & Zaharia (2021). *Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics.* CIDR 2021.
3. Akidau et al. (2015). *The Dataflow Model.* PVLDB 2015.

Case Study

Enterprise Lakehouse Migration — how Pinterest (300+ PB), Comcast (100 TB/day) and Databricks replaced their Lambda-architecture stacks with a

Paper 1 — Armbrust et al. (2020): Delta Lake

Evolution of Analytics Architectures



The Fundamental Problem (2015–2020)

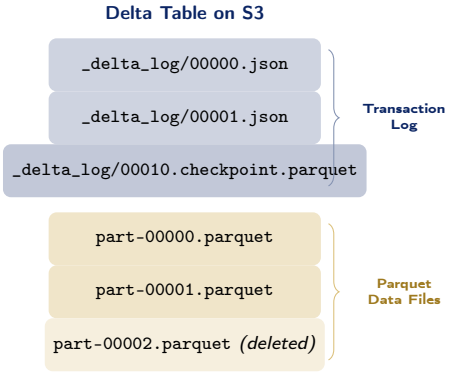
Enterprises ran **two separate systems**: a data lake (S3 + Spark) for ML workloads and a data warehouse (Redshift / Snowflake) for BI. This doubled ETL pipelines, storage cost, and data consistency risk. Delta Lake was built to eliminate this split.

Delta Lake: Publication Context & Core Mechanism

Full Citation

Armbrust, M., Das, T., Willis, J., Zhu, S., Condie, T., Zheng, S., Xin, R., & Zaharia, M. (2020). **Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores.** *Proceedings of the VLDB Endowment, 13*(12), 3411–3424.

Venue: VLDB 2020 — Rank A* (CORE)
Organization: Databricks / Apache Software Foundation
Open source: Apache License 2.0 at `delta.io`; native to Apache Spark



The Core Insight

Cloud object stores (S3, Azure Blob, GCS) are **not file systems**: they lack atomic renames, directory

Technical Contribution 1 — ACID via the Delta Log

How ACID is Achieved

Each **transaction** (insert, update, delete, merge) appends exactly one JSON entry to the Delta Log recording:

- **Add** actions: new Parquet files written
- **Remove** actions: old files logically deleted
- **Metadata** changes: schema updates, config
- **Protocol** version: minimum reader/writer version

Atomicity: a transaction either appends its log entry completely or not at all (S3 object PUT is atomic).

Isolation: OCC — concurrent writers try to append the same log entry number; only one succeeds (S3 conditional PUT); the loser retries or fails.

Time Travel & Schema Features

Time travel (data versioning): Because every version of the table is preserved in the log, Delta Lake supports reading any past snapshot:

- `VERSION AS OF 42`: read table at version 42
- `TIMESTAMP AS OF '2024-01-01'`: point-in-time query
- Enables **audit trails**, **rollback**, and **reproducible ML experiments** (lock a training dataset to a specific version)

Schema Enforcement & Evolution

- **Schema enforcement**: rejects writes that don't match the declared schema —

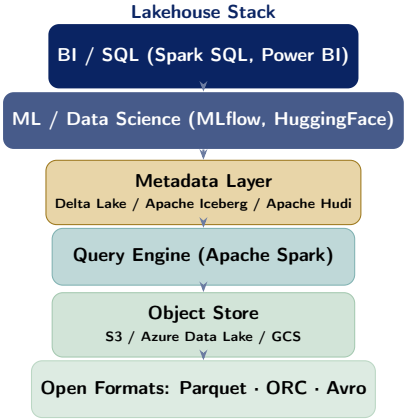
Paper 2 — Zaharia et al. (2021): Lakehouse

Lakehouse: Publication Context & Architecture

Full Citation

Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). **Lakehouse: A New Generation of Open Platforms That Unify Data Warehousing and Advanced Analytics.** *11th Annual Conference on Innovative Data Systems Research (CIDR 2021).*

Venue: CIDR 2021 — premier systems vision paper venue
Organization: Databricks
Significance: First formal architectural definition of the Lakehouse paradigm; cited by Snowflake, Google BigLake, and AWS Lake Formation design documents



The Lakehouse Definition

A Lakehouse is a data management system that

Lakehouse vs. Data Warehouse vs. Data Lake

Feature	DW	Lake	Lakeh
Storage cost	High (proprietary)	Low (S3)	Low (S
Open format	No	Yes	Yes
ACID txns	Yes	No	Yes
Schema enforce.	Yes	No	Yes
BI / SQL	Yes	Partial	Yes
ML / streaming	No	Yes	Yes
Time travel	No	No	Yes
Single copy	No (ETL copy)	N/A	**Yes**

The "No ETL Copy" Advantage

Competing Open Table Formats

The Lakehouse paper catalysed an ecosystem of open metadata formats:

- **Delta Lake** (Databricks / Apache): native Spark; strongest adoption; Z-ordering; CDC support
- **Apache Iceberg** (Netflix origin): vendor-neutral; hidden partitioning; partition evolution; Flink + Trino + Spark support
- **Apache Hudi** (Uber origin): record-level upserts; incremental processing; CDC out-of-box

2023–24 convergence: Delta and Iceberg are adopting a shared **UniForm** metadata

Paper 3 — Akidau et al. (2015): The Dataflow Model

Dataflow Model: Unifying Batch and Stream Processing

Full Citation

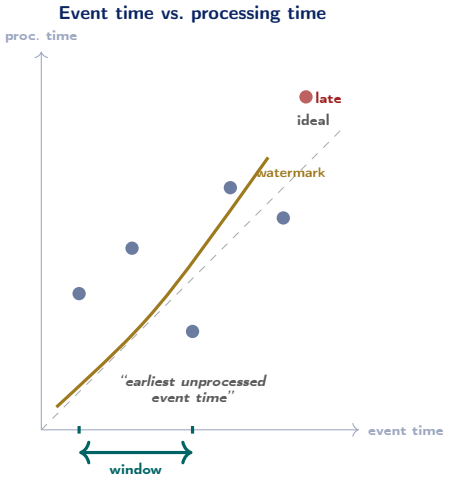
Akidau, T., Bradshaw, R., Chambers, C., et al. (2015). **The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing.** *Proceedings of the VLDB Endowment, 8*(12), 1792–1803.

Venue: PVLDB 2015 — VLDB Best Paper
Organization: Google
Open source: Led to **Apache Beam** (2016); runs on Spark, Flink, Google Dataflow backends

The Four Questions (The "Four Ws")

Any data processing pipeline can be described by:

1. **What** results are computed? (transformations:



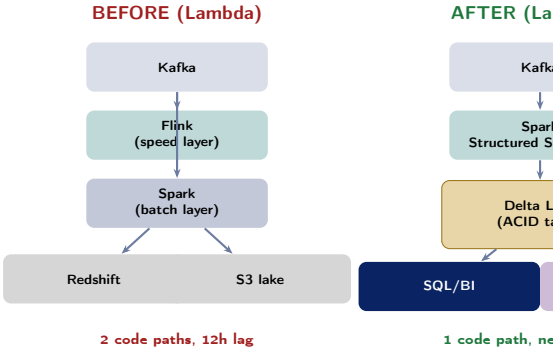
Case Study — Enterprise Lakehouse Migration in Production

From Lambda Architecture to Lakehouse

The Problem: Lambda at Scale

By 2018–2020, large enterprises running Lambda architecture faced compounding pain:

- **Pinterest (300+ PB):** two code paths (Spark batch + Flink stream) for every metric — engineers spent 40% of time keeping them in sync
- **Comcast (100 TB/day):** data copied from S3 (lake) to Redshift (DWH) nightly, creating a 12-hour data freshness gap for analysts
- **Generic pattern:** 3–5 ETL pipelines per business metric; data quality issues discovered hours after ingestion; ML feature store diverged from BI tables



Measured Outcomes & Key Lessons

Production Results (Composite from Public Reports)

Metric	Before	After
ETL pipeline count (per metric)	3–5 jobs	1 job
Data freshness (BI)	12 hours	<15 min
Storage cost (per TB)	$230 DWH	$23 S3
ML experiment reproducibility	Manual	Versioned
Failed jobs due to bad schema	Weekly	Blocked at ingest
Analyst query wait (P95)	8 min	45 sec

Sources: Pinterest Engineering Blog (2021); Comcast

Key Lessons Learned

1. **Migration, not big-bang replacement**: Lakehouse teams migrated one table at a time, running Delta Lake and legacy DWH in parallel until each table validated — zero downtime
2. **Schema enforcement first**: teams that enabled schema enforcement on day 1 found and fixed 90% of data quality bugs within the first week; teams that deferred it spent months debugging corrupted tables
3. **Time travel as an audit tool**: Compliance teams adopted `VERSION AS OF` queries to answer "what did the table look like on December 31?" —

Discussion Questions — Course Synthesis

Technical Questions (CLO4 — Apply/Analyse)

1. **ACID on S3**: Delta Lake achieves atomicity by treating S3 object PUT as atomic. Identify two scenarios where this guarantee can still break (hint: S3 eventual consistency pre-2020, multi-region buckets) and explain how Delta Lake mitigates them. *[CLO4 — Analyse]*
2. **Dataflow Four Ws**: A payment fraud detection pipeline receives transactions with a 30-second event-time lag (mobile app buffers). Define the four Ws for a tumbling 5-minute window that emits results when the watermark passes the window end AND allows late arrivals up to 2 minutes. Which Apache Beam trigger policy would you use? *[CLO4 — Apply]*
3. **Lakehouse vs. Snowflake**: Your CTO argues

Capstone Design Question (CLO4 — Design)

Design a Lakehouse for a Bangladeshi national health analytics platform:

- **Sources**: 300 hospitals (HL7 FHIR), pharmacy chains, national disease registry (CDC-style)
- **Requirements**: HIPAA-equivalent privacy; near-real-time dengue/cholera alert generation; long-term epidemiology research access; Prophet-based seasonal disease forecasting

Specify:

1. Ingestion layer (Kafka topics, connectors)
2. Storage layer (Delta Lake table design, partitioning)

Live Exercise

Live Exercise — Delta Lake ACID Operations

Task (30 minutes, pairs)

Explore Delta Lake's core ACID features using **PySpark + delta-spark** locally:

1. Write a Parquet-format DataFrame as a Delta table.
2. Append new rows (atomic transaction).
3. Perform an **UPDATE** (changes logged in Delta Log).
4. Read the table **at version 0** to verify time travel.
5. Run `DESCRIBE HISTORY` to inspect the full transaction log.
6. Break schema enforcement: try inserting a row with a wrong column type and observe the error.

Python Skeleton

```
from pyspark.sql import SparkSession
from delta import configure_spark_with_delta_pip

builder = (SparkSession.builder
           .appName("DeltaDemo")
           .config("spark.sql.extensions",
             "io.delta.sql.DeltaSparkSessionExtension")
           .config("spark.sql.catalog.spark_catalog",
             "org.apache.spark.sql.delta.catalog"
             ".DeltaCatalog"))

spark = configure_spark_with_delta_pip(
          builder).getOrCreate()

# -- Create Delta table --
df = spark.createDataFrame([
  (1,"Alice",30),(2,"Bob",25)],
  ["id","name","age"])
df.write.format("delta").save("/tmp/demo_table")

# -- Append (new transaction) --
df2 = spark.createDataFrame([
  (3,"Carol",35)], ["id","name","age"])
```

References

Seminal Papers

- Armbrust, M., et al. (2020). Delta Lake: High-performance ACID table storage over cloud object stores. *PVLDB, 13*(12), 3411–3424.
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *CIDR 2021*.
- Akidau, T., et al. (2015). The Dataflow model. *PVLDB, 8*(12), 1792–1803.

Textbooks [TW], [SA], [TE]

- **[TW]** White, T. (2015). *Hadoop: The Definitive Guide*, 4th ed. O'Reilly.
- **[SA]** Acharya, S., & Chellappan, S. (2015). *Big Data and Analytics* (Ch. 12). Wiley.
- **[TE]** Erl, T., et al. (2016). *Big Data Fundamentals* (Ch. 13). Prentice Hall.

Case Study Sources

- Pinterest Engineering Blog (2021). Building Pinterest's new wide column storage system.

Closing — Week 11 & Course Synthesis

Course Technology Stack — Full Picture

Weeks	Technology & Concept
1–2	Big Data fundamentals, 3Vs, integration challenges
3–4	HDFS, MapReduce, HBase, Pig, Hive
5	YARN, Parquet/ORC/Avro, optimization
6	Spark RDDs, DataFrames, lazy evaluation
7	HiveQL, SQL-on-Hadoop, Presto
8	Kafka, Lambda/Kappa, data pipelines
9	Visualization: Tufte, D3, Tableau,

Quote of the Course

"The exciting thing about the Lakehouse is that it's not a compromise — it's actually better than either a data warehouse or a data lake at their own jobs."

— Matei Zaharia, co-creator of Spark & Apache Delta Lake

What Comes Next

- **Project exhibition**: deploy your end-to-end big data pipeline (CLO4)
- **Career paths**: Data Engineer, Data Scientist, Big Data Architect, ML